Лабораторийн ажил 4

FoodFlow архитектурын баримт

МУИС, МТЭС, МКУТ, Програм хангамж

Ю. Төгөлдөр, 23B1NUM2350

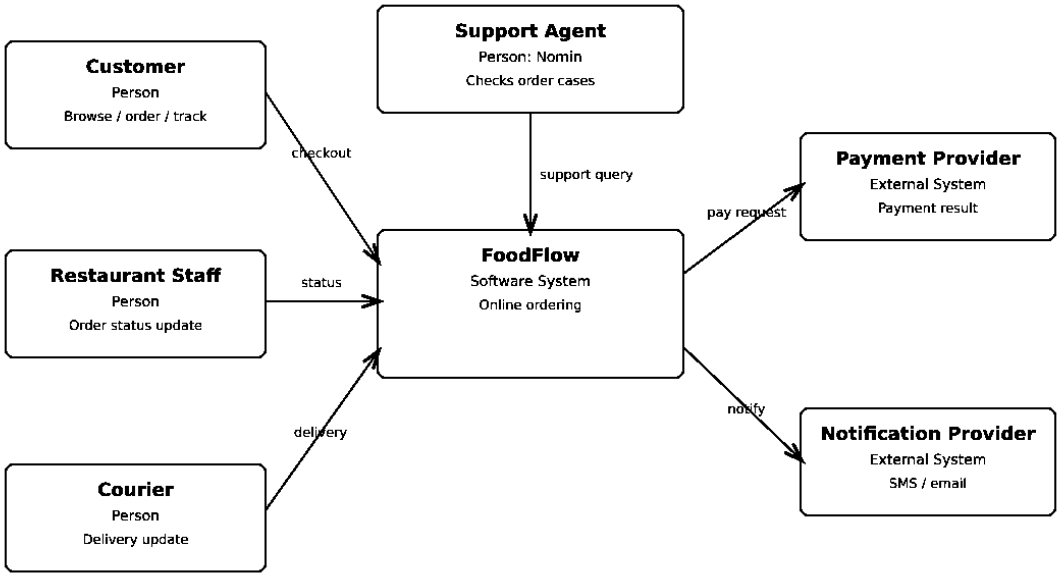
Танилцуулга ба зорилго

FoodFlow нь хэрэглэгчид хоол сонгох, сагс бүрдүүлэх, хүргэлтийн хаяг болон төлбөрийн аргаа өгөх, захиалгын төлөвөө харах боломжтой вэб систем. Рестораны ажилтан захиалгыг боловсруулж, хүргэлтийн ажилтан явцын мэдээлэл өгнө.

Сонирхогчид: Саруул (хоол захиалагч), Ану (frontend хөгжүүлэгч), Билэгт (backend хөгжүүлэгч), Номин (support), restaurant staff, courier, QA. Ану, Билэгт, Номин нь W01 persona-д тодорхойлогдсон. Стек ба үйлчилгээ сонгосон шалтгааныг ADR-001–003-т бичсэн.

Хамрах хүрээ ба орчин

Системийн хил: FoodFlow-ийн Web App, API, worker, PostgreSQL, Redis бидний хариуцах хэсэг. Payment Provider төлбөрийн хүсэлт болон үр дүнг, Notification Provider мэдэгдлийн хүргэлтийг гаднаас гүйцэтгэнэ. Картын raw мэдээлэл FoodFlow-д хадгалагдахгүй. Гаднын үйлчилгээний нэр, жинхэнэ URL, гэрээ баталгаажаагүй тул ерөнхий нэр хэрэглэсэн.



FoodFlow C4 Context

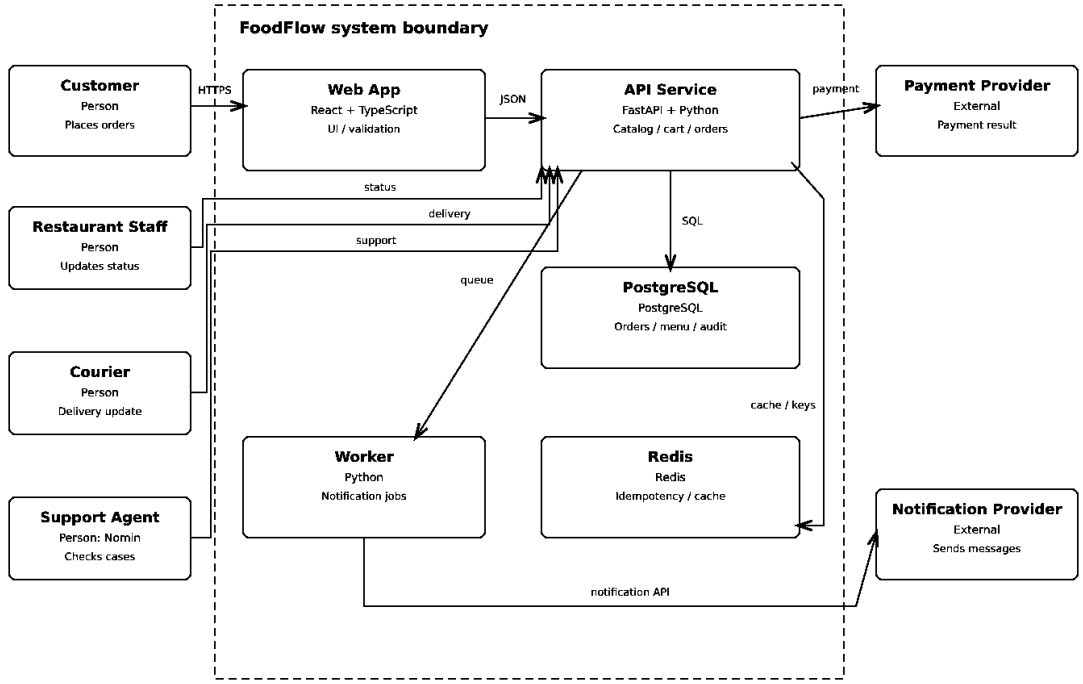
Сум тус бүрт хэн хэнд ямар мэдээлэл явуулж буйг бичсэн. W01-ийн Ану API хэрэглэгч учраас diagram-д байгаа хүн бүрийг Ану гэж нэрлээгүй; тэр Web App ба API-ийн интерфейсийг хэрэглэх хөгжүүлэгч юм.

Гаднах хүн эсвэл систем	Холбоо ба мэдээллийн урсгал	Persona / шаардлага
Саруул, Customer	Хоол, сагс, checkout, өөрийн order-ийн төлөв уншина	W01-ийн end-end user; FR-01–15.

Гаднах хүн эсвэл систем	Холбоо ба мэдээллийн урсгал	Persona / шаардлага
Restaurant Staff	Шинэ order үзэж Confirmed, Preparing төлөвийг шинэчилнэ	W01-ийн Номины мэдэх ёстой төлөвийн эх үүсвэр; FR-12.
Courier	Хүргэлтийг авснаа болон Delivered төлөвийг дамжуулна	Номины хэрэглэгчид тайлбарлах төлөв; FR-12.
Номин, Support	Зөвшөөрөгдсөн order lookup, алдааны мэдээллээр хэрэглэгчид хариулна	W01-ийн support persona; FR-12/14, NFR-03.
Payment Provider	FoodFlow төлбөрийн хүсэлт явуулж, provider төлбөрийн баталгаажилтын token/result буцаана	FR-10/11; raw card мэдээлэл дамжуулахгүй.
Notification Provider	Worker event болон хүлээн авагчийн destination илгээж, delivery result авна	FR-15; алдаа бол retry queue.

Хил дээрх дүрэм: Provider webhook-д signature болон давхардлыг шалгана; API ажилтны role болон customer-ийн order ownership-ийг шалгана. Timeout болсон төлбөрийг сохроор амжилттай гэж үзэхгүй. Энэ нь W03-ийн NFR-02/03, FR-11/12-той холбогдоно.

5 Бүрэлдэхүүн хэсгийн харагдац



FoodFlow C4 Container

C4 Container нь deploy/run хийх үүргийн хил; түүний доторх классын диаграм биш. FoodFlow системийн хил дотор таван container байна (хязгаар ≤8).

Container	Технологи	Гол хариуцлага, эзэмших өгөгдөл	Гол харилцаа
Web App	React, TypeScript; browser	Menu/Cart/Checkout/Status/History UI; keyboard navigation	Customer, staff, courier, support → HTTPS → API.
API	FastAPI, Python; REST/JSON	Хандах эрх, каталог, cart, price, order, status; provider integration-ийн зохицуулалт	Web App, PostgreSQL, Redis, Payment Provider.
Worker	Python background worker	Event-ээс notification бэлтгэх, retry, provider result тэмдэглэх	Redis queue, PostgreSQL, Notification Provider.
PostgreSQL	Relational database	Menu, cart/order, ownership, payment result, status history-ийн durable өгөгдөл	API, Worker.
Redis	Queue, idempotency key cache	Ажил гүйцэтгэх queue, давтан POST-ийн key-ийн 24 цагийн бүртгэл	API, Worker; durable order PostgreSQL-д.

5.1 Дотоод урсгал: Web App API-д token-тай хүсэлт илгээнэ. API owner/role-г шалгаад PostgreSQL-д өгөгдлийг бичиж уншина. POST /orders-д idempotency key-г Redis-ээр шалгаад нэг order transaction-г PostgreSQL-д баталгаажуулна; баталгаажсан төлбөрийн result-ийг бүртгэнэ. Notification ажил Redis queue-д орж Worker гаднах provider-т илгээнэ. Redis түр тасарвал order write аюулгүй үргэлжлэх fallback нь хөгжүүлэлтийн шийдвэр болон туршилтаар тодорхойлогдох ёстой; одоогоор fail-closed байж duplicate үүсгэхгүй гэж төлөвлөсөн.

5.2 W03 SRS-той мөрдөх холбоо. Доорх багана нь шаардлагыг хариуцах үндсэн блокийг заана; олон блок дамнасан хэсгийг хавсаргасан. “Mapped” гэдэг нь баримтын хамрах хүрээ, хэрэгжүүлсэн гэдгийн баталгаа биш.

SRS ID	Хариуцах building block	Ямар үүргээр холбогдох вэ	Төлөв
FR-01	Web App, API, PostgreSQL	Каталог харуулах, унших, хадгалах	Mapped
FR-02	Web App, API, PostgreSQL	Хайлт, ангиллын шүүлт	Mapped
FR-03	Web App, API, PostgreSQL	Орц, харшил, тайлбар	Mapped
FR-04	Web App, API, PostgreSQL	Сагсанд item нэмэх	Mapped
FR-05	Web App, API, PostgreSQL	Quantity, subtotal шинэчлэх	Mapped
FR-06	Web App, API, PostgreSQL	Cart item устгах	Mapped
FR-07	Web App, API	Үнийн томьёог серверээр тооцож UI-д харуулах	Mapped
FR-08	Web App, API, PostgreSQL	Хаяг validate, order draft	Mapped
FR-09	Web App, API, PostgreSQL	Боломжтой хүргэлтийн цонх	Mapped
FR-10	Web App, API, Payment Provider	Төлбөрийн арга сонголт, хүсэлт	Mapped
FR-11	Web App, API, Redis, PostgreSQL, Payment Provider	Нэг order; key, transaction, payment result	Mapped
FR-12	Web App, API, PostgreSQL	Өөрийн order-ийн одоогийн төлөв	Mapped
FR-13	Web App, API, PostgreSQL	Төлөвөөс шалтгаалсан цуцлалт	Mapped
FR-14	Web App, API, PostgreSQL	Зөвхөн өөрийн order history; support эрх	Mapped
FR-15	API, Redis, Worker, Notification Provider	Order event ба мэдэгдэл, retry	Mapped
NFR-01	Web App, API	Mobile LCP хэмжилт	Mapped
NFR-02	API, Redis, PostgreSQL	Давтан request ба нэг захиалга	Mapped
NFR-03	API, PostgreSQL	Role, owner, audit	Mapped
NFR-04	Web App, API, Worker	Uptime probe ба provider fault handling	Mapped
NFR-05	Web App	Keyboard controls-ийн бүрэн урсгал	Mapped

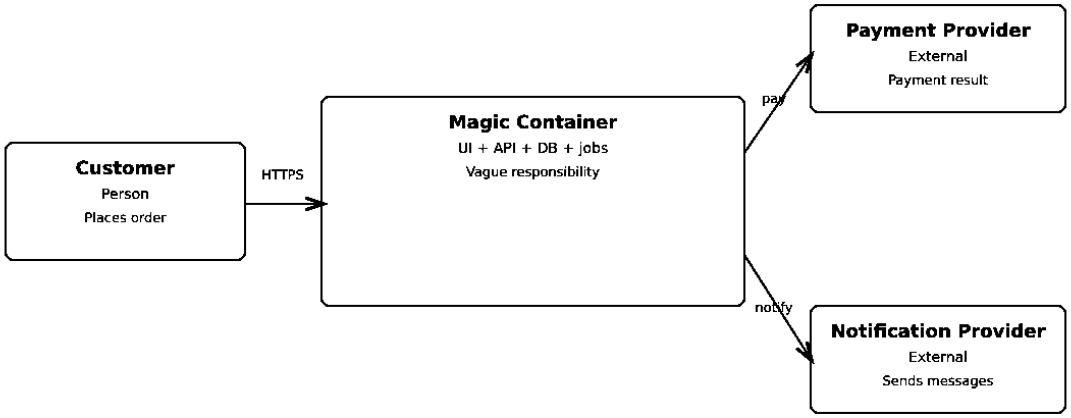
6 Шийдвэрийн тэмдэглэлийн индекс

ADR	Сэдэв	W01 persona-ийн асуудал	Төлөв
ADR-001	React + TypeScript Web App	Ану: UI/API жишээ, version тодорхой байх	Proposed
ADR-002	PostgreSQL + Redis-ийн ялгаа	Билэгт: давхар order, хадгалалтын хил	Proposed
ADR-003	Versioned REST/JSON, алдааны гэрээ	Ану: auth/error; Номин: тайлбарлах алдаа	Proposed

ADR-ийн Proposed төлөв нь баг шийдвэрийг албан ёсоор зөвшөөрсөн гэж худал мэдүүлэхгүй. Төслийн workspace-д хэвлэгдсэн эсэх нь гаднын шалгах ажил.

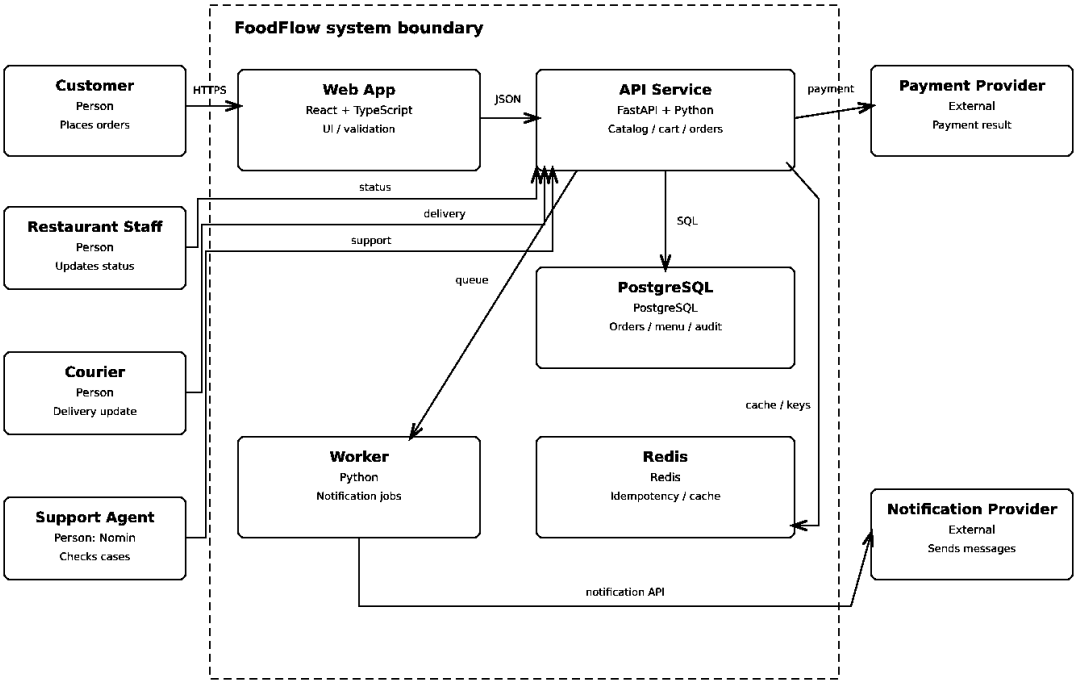
7 UE-4 Magic Container-ийг засах нь

Эхний ноорогт “Magic Container” гэх нэг хайрцаг UI, API, төлбөр, DB, мессеж, retry бүгдийг хариуцсан. Уншигч өгөгдөл хаана үлдэх, provider-ийн алдааг хэн засах, хоёр дахь POST давхар order болгох эсэхийг мэдэхгүй.



UE-4 өмнөх Magic Container

Шинэ зурагт хэрэглэгчийн дэлгэц, API, worker, PostgreSQL, Redis үүргээрээ салж, гаднах provider-ууд урсгалаа тайлбартай авсан.



UE-4 дараах контейнерүүд

(1) хэрэглэгчийн order аяллыг цаасан дээр эхлэн зурж хаанаас уншихыг тогтоов;

(2) өгөгдлийн owner, provider boundary, retry-г тус тусад нь нэрлэв;

(3) сумыг HTTPS, SQL, queue, provider event гэж яг мэдээллээр нь шошголов. Нэг нүүрийн ADR-004 тайлбар хавсаргасан.

Шалгах жагсаалт ба эргэцүүлэл

Definition of Done	Бодит байдал
arc42 1, 3, 5; Context ба Container diagram-	Энэ repo-ийн Markdown, Mermaid, SVG, PNG-д бэлэн. Course workspace-д байршуулах ажил

Definition of Done	Бодит байдал
as-code	гаднаас хийгдэнэ.
Гурван MADR ба persona-тай холбоос	ADR-001–003 бичигдсэн, Proposed төлөвтэй; workspace-д нийтэлснийг мэдүүлээгүй.
FR-үүдийн ≥80% traceability	15/15 = 100% документын зураглал; runtime verification тусдаа.
UE-4 before/after ба нэг нүүр ADR	Зураг, эх код, ADR-004 хавсаргасан.

Standup: (1) arc42-ийн гурван хэсэг нэмэгдсэн; (2) C4 хоёр диаграм Mermaid эх кодтой; (3) payment provider-ийн бодит contract багийн баталгааг хүлээнэ. Retrospective: (1) §5-ийн traceability хамгийн хурдан хэрэглэгдэх үнэ цэнтэй: orphan шууд харагдана; (2) таван container хангалттай түвшин; (3) ADR бүрт баталгаажуулах хүн, огноог дараагийн sprint-д нэмнэ.

ADR-001 React ба TypeScript веб интерфейс

Status: Proposed. Date: 2026-09-28. Owner for review: frontend ба backend баг. Холбоо: W01 persona Ану; FR-01–10, FR-12–14, NFR-05; W04 arc42 §5.

Context

Ану Menu → Cart → Checkout урсгалыг API-тай холбохдоо request, response, error-ийн хэлбэрийг таах шаардлагатай гэж persona ноорогт тэмдэглэсэн. Саруулын гар дээр алдааг ойлгомжтой харуулахын тулд дэлгэцийн төлөвүүдийг тогтвортой удирдах хэрэгтэй. Keyboard хэрэглээ NFR-05-ийн хэмжүүртэй.

Decision

Browser-д ажиллах Web App-ийг React + TypeScript-ээр зохион байгуулна. API client нь /api/v1 гэрээг нэг модульд хэрэгжүүлж, code/message алдааг UI-ийн тайлбарлах төлөвт хөрвүүлнэ. Cart, Checkout, Status view-д accessibility-ийн keyboard дарааллыг component бүрээр шалгана. Энэ бол төсөлд санал болгож буй сонголт; framework-ийг аль хэдийн суулгасан гэсэн мэдэгдэл биш.

Alternatives considered

Server rendered template нь бага JavaScript-тай ч cart-ийн олон алхамт state-г client-д удирдах шийдэл шаарддаг. Plain JavaScript нь жижиг эхлэлтэй ч API талбарын зөрүүг compile time-д барих боломж сул.

Consequences

Эерэг: typed API client нь Анугийн auth/error зөрөөг эрт илрүүлэх боломж өгнө, checkout view-г component-оор салгана. Сөрөг: build pipeline, dependency update, frontend тестийн ажил нэмэгдэнэ. Шалгах нь NFR-05 keyboard test болон FR-11-ийн давтан захиалгын UI acceptance test. Зөвшөөрөгдвөл owner version-ийг lockfile ба README-д тэмдэглэнэ.

ADR-002 PostgreSQL-д durable order, Redis-д түр queue ба key

Status: Proposed. Date: 2026-09-28. Owner for review: backend ба operations баг. Холбоо: W01 persona Билэгт; FR-04–15, NFR-02/03/04; W04 arc42 §5.

Context

Билэгтийн асуудал нь давхар POST нэгээс олон order үүсгэх болон аль өгөгдөл тогтвортой хадгалагдахыг таах явдал. Order status, ownership, payment result нь сэргээх шаардлагатай өгөгдөл; мэдэгдэл бол дахин оролдож болох ажил. Ижил Idempotency-Key-ийн 24 цагийн дүрмийг W01 README-ийн засварт бичсэн.

Decision

Order, item, ownership, status transition, payment result-ийг PostgreSQL transaction-д хадгална. Redis-д background job queue болон 24 цагийн idempotency key lookup хадгална. Order-ийн үнэн эх сурвалж

PostgreSQL; Redis-ийн cache-д л order-ийг хадгалахгүй. Duplicate-ийг database unique constraint болон application key check хоёул хамгаалах нарийвчилсан schema-г хэрэгжүүлэлтийн өмнө батална.

Alternatives considered

Зөвхөн Redis ашиглавал түр хадгалалтын алдагдал, owner lookup ба transaction-ийн асуудал үүснэ. Зөвхөн PostgreSQL-д queue, key бүгдийг хадгалж болно; эхний хувилбарын ачаалал, retry-ийн загварыг харьцуулж байж эцэслэнэ.

Consequences

Эерэг: durable захиалга болон түр ажил тусдаа, FR-11 тестийн нэг order нөхцөлийг тодорхой шалгаж болно. Сөрөг: хоёр datastore-ийн monitoring, consistency, Redis outage-ийн fallback төлөв хэрэгтэй. Fail-closed үед захиалгыг түр татгалзуулах боловч duplicate үүсгэхгүй; энэ бодлого багийн шалгалтад нээлттэй. QA ижил key-ийн хоёр POST, PostgreSQL restart, Redis outage-ийг хамтад нь туршина.

ADR-003 /api/v1 REST/JSON ба тогтмол алдааны гэрээ

Status: Proposed. Date: 2026-09-28. Owner for review: frontend, backend, support баг. Холбоо: W01 persona Ану ба Номин; FR-01–15, NFR-03; W04 arc42 §3 ба §5.

Context

Анугийн auth header болон success/error JSON зөрөх хүндрэл, Номины хэрэглэгчид алдааг тайлбарлах хэрэгцээ нь нэг интерфейсийн гэрээ шаарддаг. Захиалгыг дахин илгээхэд FR-11-ийн duplicate=0 баталгааг API түвшинд уншихуйц байлгах ёстой.

Decision

FoodFlow Web App нь HTTPS дээр /api/v1 REST/JSON ашиглана. Нэвтэрсэн request-д Authorization: Bearer token, POST /orders-д Idempotency-Key байна. Амжилттай response-д resource болон тогтмол order_id; алдаанд code, message, шаардлагатай үед field өгнө. 401 auth, 403 эрх, 404 байхгүй resource, 409 төлөвийн зөрчлийг ялгана. OpenAPI-г contract-ийн нийтлэх хэлбэр болгон version-той нь хадгална.

Alternatives considered

Endpoint бүр өөр хэлбэрийн алдаа буцаах нь богино хугацаанд амар боловч UI ба support-д зөрүү үүсгэнэ. GraphQL нь сонгомол query-д ашигтай ч захиалгын төлөв, идемпотент write-ийн эхний гэрээг илүү төвөгтэй болгоно.

Consequences

Эерэг: Ану request/response-ийг нэг reference-ээс олно, Номин code-д тулгуурлан дараагийн алхам хэлнэ. Сөрөг: schema version, migration, алдааны code бүрийг өөрчлөх хяналт шаардана. FR-11/12/14 contract test, 401/403/404/409-ийн UI test шаардлагатай. Provider-ийн webhook-ийн жинхэнэ schema-г гэрээ баталгаажсаны дараа тусад нь нэмнэ.

ADR-004 UE-4 Magic Container-ийг үүргээр салгах

Status: Proposed for exercise. Date: 2026-09-28. Owner for review: FoodFlow хөгжүүлэлтийн баг. Холбоо: W01 Ану/Билэгт/Номин; FR-11/12/15; W04 UE-4.

Context

Эхний зураг-т “Magic Container” нэртэй нэг хайрцаг customer UI, order logic, payment, database, notification-ийг зэрэг хариуцах мэт зурсан. Хэрэглэгчээс эхлэн уншвал төлбөрийн баталгаа хаанаас ирэх, алдаа хаана

хадгалагдах, retry-г хэн ажиллуулах тодорхойгүй. Билэгтийн давхар order-ийн асуудал, Номины төлөв тайлбарлах хэрэгцээг зураг хангахгүй.

Decision

Дараах зурагт Web App, API, Worker, PostgreSQL, Redis гэсэн таван контейнерийг нэр, технологи, нэг гол үүргээр нь салгасан. Payment Provider, Notification Provider хоёрыг системийн хилээс гадна тавьсан. Сум тус бүрийн HTTPS/JSON, SQL, queue, event шошго нь уншигч юу шилжиж байгааг хэлнэ. Эхлээд хэрэглэгчийн захиалга өгөх аяллыг цаасан дээр тогтоогоод, дараа нь owner-ийг бичиж, төгсгөлд холбоосуудад нэр өгсөн

Alternatives considered

Magic Container-ийг хэвээр үлдээвэл хамгийн жижиг зураг болох ч хариуцлага, алдаа, өгөгдлийн owner-ийг нууж орхино. UI/API/DB гэсэн гурван хайрцаг нь notification retry болон key queue-ийн үүргийг тодорхой гаргахгүй.

Consequences

Эерэг: FR-11-ийн key ба durable order тусдаа, FR-15-ийн retry Worker-т, FR-12-ийн status PostgreSQL-д байгааг уншигч харна. Номин status-ийн өгөгдлийн эх үүсвэрийг, Ану UI/API гэрээг хурдан олно. Сөрөг: зурган дээр илүү олон сум гарна; эх кодыг өөрчлөлт бүрд шинэчлэх сахилга шаардана. Хожмын W05-д provider webhook schema, Redis outage fallback, team approval-ийг шийднэ. Энэ ADR нь сургалтын нэг нүүрийн баримт бөгөөд бодит deployment-ийн баталгаа биш.